AUTOMATED WEB SECURITY SCANNER USING PYTHON

Project Report

Project Area: Cybersecurity / Web Application Security

Technology: Python, Requests library, HTTP response headers

Prepared as an educational project demonstrating basic, read-only web security checks.

Ethical-use statement: This tool must only be used on websites owned by the user or where explicit permission has been granted.

1. Abstract

This project implements a small Python command-line tool that makes a normal HTTP GET request to an authorized website and reports selected security-related observations. It checks whether the URL uses HTTPS, looks for common security response headers, observes the Access-Control-Allow-Origin header, and records the HTTP status and final URL. It is intended for learning and awareness, not as a replacement for a professional security assessment.

2. Problem Statement

Website owners need simple ways to understand some basic web security settings. Beginners may find security headers and HTTPS behavior difficult to inspect manually. This project automates a small set of observations and prints them in a readable report.

3. Objectives

• Check whether the entered URL uses HTTPS.
• Inspect common response headers: Strict-Transport-Security, Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, Referrer-Policy, and Permissions-Policy.
• Observe the Access-Control-Allow-Origin header without sending crafted origins or attempting access to protected resources.
• Display response status and redirect destination.
• Optionally save results in JSON format.

4. Tools and Technologies

Python 3.9 or later is used for the program. The Requests library performs the HTTP GET request and reads the response headers. The argparse module processes command-line options, and the json module saves results for later review.

5. Methodology

1. The user enters a URL that they own or are authorized to test.
2. The program normalizes the URL by adding HTTPS when no scheme is supplied.
3. It performs a single GET request with a timeout and follows normal redirects.
4. It records the final URL, HTTP status, and selected response headers.
5. It prints each observation with a simple status such as PASS, INFO, REVIEW, WARNING, or ERROR.
6. If requested, it saves the results as a JSON file.

6. Main Modules

URL validation: checks that the input contains a hostname.
HTTPS observation: checks the URL scheme and whether the final URL uses HTTPS.
Security-header inspection: checks whether selected headers are present and shows their values.
CORS observation: reports the Access-Control-Allow-Origin value, if returned.
Report output: displays readable results and optionally writes JSON.

7. How to Run

Install Python 3.9 or later and install dependencies with `python -m pip install -r requirements.txt`. Run the tool with `python scanner.py https://example.com`, replacing the example with an authorized target. To save JSON output, use `python scanner.py https://example.com --json report.json`.

8. Expected Output

The program prints the target URL, final URL and HTTP status when available, followed by each check and its observation. A missing header is labelled INFO rather than automatically labelled as a vulnerability. If the request fails, the program reports an ERROR and the request failure reason.

9. Limitations

This is a basic scanner. It does not crawl pages, test authentication, inject payloads, exploit vulnerabilities, assess every CORS scenario, or provide a complete TLS audit. Header requirements vary by application. A missing header or observed CORS value requires context and does not alone prove a vulnerability.

10. Ethical and Safety Considerations

Use the scanner only on systems you own or have explicit permission to assess. Do not use the results to claim a website is vulnerable without validation. Avoid collecting private data and follow the target organization's testing rules.

11. Future Scope

Possible improvements include unit tests, cookie attribute checks, certificate-expiry checks, HTML/PDF report generation, a graphical interface, and a more detailed explanation of each finding.

12. Conclusion

The project demonstrates how Python can automate basic observations of HTTPS and web response headers. It provides a small starting point for learning web security, HTTP behavior, and responsible security testing. It should be extended and tested before use in any real security workflow.